

Forelesningsnotater Introduksjon til Maskinl ring – TDT4172

Inga Str mke

September 1, 2026

Contents

1	Veiledet l�ring	2
1.1	Data	2
1.2	Logistisk regresjon	4
1.2.1	Trening og tapsfunksjon	5
1.2.2	Gradient descent	7
1.2.3	Trening	8
1.2.4	Evaluering	9
1.2.5	Klassifiseringsterskel	11
1.2.6	Kort om entropi og cross entropy loss	12
1.2.7	Bayes' teorem	13
1.2.8	Ikke-uavhengige features	15
1.2.9	Dimensjonsforbannelsen	15
1.2.10	Trening p� ubalanserte data	16
1.3	Beslutningstr�r	17
1.3.1	Noder	17
1.3.2	Gini Impurity	18
1.3.3	Entropi	18
1.3.4	Bygge beslutningstr�r	19

1 Veiledet læring

1.1 Data

Maskinlæring går ut på å programmere datamaskiner slik at de kan lære fra data. Arthur Samuel (1959) ga definisjonen

“Machine learning is the field of study that gives computers the ability to learn without being explicitly programmed.”

Tom Mitchell (1997) ga den mer presise definisjonen

“A computer program is said to learn from experience E with respect to some task T and some performance measure P , if its performance on T , as measured by P , improves with experience E .”

Utgangspunktet er altså *experience*, representert i form av data, som brukes av et dataprogram hvis *performance*, altså ytelse, øker på en *task*, altså oppgave. Data har vi i form av datasett, som består av datapunkter med ulike egenskaper. Disse egenskapene omtales som *variabler* innen statistikk, og ofte brukes det engelske begrepet *features* i maskinlæringssammenheng. Vi vil bruke disse to begrepene om hverandre. Variabler, eller features, kan være kvalitative eller kvantitative, se eksempler i tabell 1. *Kvalitative variabler*, også omtalt som kategoriske features, kan ha et tellbart antall verdier, henholdsvis beskrive et endelig antall kategorier. Disse må ofte enkodes før vi kan bruke dem i maskinlæring (for eksempel ved bruk av label encoding), for eksempel hvis de er representert som strenger i stedet for tall. *Kvantitative variabler* beskriver data som kan måles og sorteres. De har numeriske verdier og man kan utføre matematiske operasjoner på dem (addisjon, subtraksjon, osv), og de kan være både diskrete og kontinuerlige. Slike features kan i prinsippet brukes direkte i maskinlæring, men ofte bør de behandles først, for eksempel gjennom normalisering eller mer avansert feature engineering, som vi skal se på senere.

Table 1: To typer features for maskinlæringsmodeller.

	Kvalitative variabler	Kvantitative variabler
Representerer	Kategorier	Numeriske verdier
Eksempler	Sjanger {action, komedie, ...}, Aldersgruppe {ung, ..., gammel}, Type frukt {eple, banan, ...}	Inntekt, alder, antall lyttere, areal, nedbørsmengde, temperatur

I et datasett representerer kolonnene ulike features, mens radene inneholder de ulike datapunktene. Når vi gjør maskinlæring ønsker vi å lage en databasert modell som estimerer en funksjonsverdi for gitte data. Hvis datasettet vi bruker inneholder en kolonne som angir funksjonsverdien for det aktuelle datapunktet, omtaler vi denne kolonnen som *target*, og vi kan gjøre veiledet læring, fra engelsk *supervised learning*.

Vi kan for eksempel bruke datasettet *Titanic* fra Kaggle¹, hvorav et utvalg er vist i tabell 2. Her representerer PassengerId, Pclass, Name, Sex og Age datasettets fem features, mens Survived representerer target – hvis målet er å lage en modell som beregner sjansen for at en person overlever.

Oppgave: (Tenk deg om før du sjekker svaret på oppgaven, på neste side)

1. Hvilke variabler er kategoriske?
2. Hvilke variabler er binære?
3. Hvilke variabler er kontinuerlige?

¹<https://www.kaggle.com/c/titanic>

Table 2: Titanic-datasettet fra Kaggle.

PassengerId	Pclass	Name	Sex	Age	Survived
1	3	Braund, Mr. Owen Harris	male	22	0
2	1	Cumings, Mrs. John Bradley	female	38	1
3	3	Heikkinen, Miss. Laina	female	26	1
4	1	Futrelle, Mrs. Jacques Heath	female	35	1
5	3	Allen, Mr. William Henry	male	35	0
6	3	Moran, Mr. James	male		0
7	1	McCarthy, Mr. Timothy J	male	54	0
8	3	Palsson, Master. Gosta Leonard	male	2	0
9	2	Montvila, Rev. Juozas	male	27	1

4. Hvilken variabel bør vi ikke bruke?

Svar på oppgaven:

1. Pclass, Name, Sex, Survived, (PassengerId, om vi anser de ulike Id'ene som hver sin kategori)
2. Sex, Survived
3. Age
4. PassengerId, Name

Grunnen til at vi ikke bør bruke PassengerId, er at denne variabelen ikke inneholder informasjon om egenskapene til en passasjer, og sannsynligvis er tilfeldig valgt. Name bør kun brukes hvis vi ønsker å lage en modell som forstår språk tilstrekkelig godt til å kunne finne en relasjon mellom typiske navn og for eksempel dyktighet i redningsbåter. For modellene vil skal lage, vil ikke Name inneholde relevant informasjon.

Før vi kan begynne på modelleringen, fjerner vi variablene vi ikke vil bruke, og innfører nyttig notasjon, se tabell 3. Hver rad i dataene representerer et datapunkt, og vi bruker indeks i oppe for å angi nummer

Table 3: Titanic-datasettet etter at vi har valgt ut hvilke variable vi vil beholde, og innført notasjon.

	x_1	x_2	x_3	y
$\mathbf{x}^{(1)}$	3	male	22	0
$\mathbf{x}^{(2)}$	1	female	38	1
$\mathbf{x}^{(3)}$	3	female	26	1
$\mathbf{x}^{(4)}$	1	female	35	1
$\mathbf{x}^{(5)}$	3	male	35	0
$\mathbf{x}^{(6)}$	3	male		0
$\mathbf{x}^{(7)}$	1	male	54	0
$\mathbf{x}^{(8)}$	3	male	2	0
$\mathbf{x}^{(9)}$	2	male	27	1

i datasettet. Eventuelt kan vi sette en parentes rundt indeksen, for å ikke forvirre indeksen med en eksponent. Hver kolonne representerer en feature, med indeks i nede. Vi brukes fet $\mathbf{x}$ for å angi en vektor, $\mathbf{x}^{(1)} = (x_1^{(1)}, x_2^{(1)}, x_3^{(1)})$. For å benevne target, altså størrelsen vi ønsker å estimere, bruker vi y .

Vi kan skrive oppgaven vi ønsker å løse som følger

$$\underbrace{(\text{Pclass, Sex, Age})}_{\text{features } \mathbf{x}^{(i)} = (x_1^{(i)}, x_2^{(i)}, x_3^{(i)})} \rightarrow \underbrace{\text{Survived}}_{\text{target } y^{(i)}} \quad (1)$$

og siden vi har en target per datapunkt, kan vi altså gjøre veiledet læring for å lage en modell som løser denne oppgaven. For å oppsummere:

- Vi har features $\mathbf{x}$ og targets y
- Vi ønsker å estimere y for nye $\mathbf{x}$, basert på relasjonene representert mellom kjente $\mathbf{x}$ og y .

Før vi begynner å lage en modell bør vi få en oversikt over sammenhenger i dataene. **Oppgave:** Se på tabell 3 igjen. Hva bør vi gjøre før vi plottet disse dataene? Svaret lyder:

- Rad 6 mangler en verdi for feature 3; dette vil gi en feilmelding når vi prøver å plote dataene,
- feature 2 er strenger (strings), altså ikke-numeriske verdier, som en datamaskin ikke kan plote uten videre,
- feature 3 er av en helt annen størrelsesorden enn features 1 og 2, og kan i verste fall gi misvisende plott.

Vi kan løse disse problemene ved hjelp av innebygget funksjonalitet i **pandas** og **sklearn**. En enkel måte å bli kvitt rader med manglende verdier er med **pandas**-metoden **dropna()**, som fjerner rader der minimum én verdi mangler. For å gjøre om feature 2 til numeriske verdier, kan vi bruke en **LabelEncoder**, som mapper hver feature-verdi til et unikt tall. I vårt tilfelle kan for eksempel “female” mappes til 0, og “male” til 1 (eller omvendt). For å skalere verdiene til feature 3 til intervallet $[0, 1]$, kan vi bruke **MinMaxScaler**. Slike teknikker, der vi fjerner rader, encoder eller skalerer feature-verdier, omtales som preprosessering av dataene.

Etter at vi er ferdige med å redigere dataene, bør vi plote dem for å se hvordan ulike features fordeler seg. Merk at plotting også kan brukes for å få idéer til hvordan dataene bør behandles, så plotting og dataprosessering kan skje om hverandre. Figur 1 viser fire ulike plott, hvorav figurene 1a og 1b viser bar-plott, og figurene 1c og 1d viser histogrammer, over ulike features og fargekoding basert på verdien til target Survived. Et *bar-plott* viser oss middelerdien per kategori, i dette tilfellet middelerdien til datapunktene som svarer til Survived=1. Vi bruker altså bar-plott for å sammenlikne variablers gjennomsnittsverdi på tvers av kategorier. Et *histogram* viser oss fordelingen av en variabel, i dette tilfellet innad i hver klasse, altså en fordeling for variabelverdier som svarer til Survived=1 og en annen for Survived=0. Stolpene representerer intervaller for variabelens verdi, og disse intervallene omtales som *bins*. Valg av bin-størrelse påvirker hvordan histogrammet ser ut, og det finnes ingen almenngyldig regel for hvor store (hvhv hvor mange) bins et histogram bør ha, siden dette er avhengig av dataenes underliggende fordeling. Hvis et histogram ser veldig hakkete ut har man ofte valgt for mange bins, som følgelig inneholder for få datapunkter, og hvis det er vanskelig å få øye på en form i fordelingen, har man ofte valgt for få bins, som følgelig inneholder for mange datapunkter. Kun ved å se på plottene i figur 1 får vi allerede en idé om at passasjerer i Pclass=1, kvinnelige passasjerer og yngre passasjerer ofte overlever. Dette passer godt med det vi vet fra filmen :).

Når vi er ferdige med å preprosessere dataene og har studert dem gjennom plott, er vi klare for å begynne med modelleringen, altså å bygge en modell. For Titanic-dataene ønsker vi å lage en modell f som estimerer sannsynligheten for å overleve, target y , basert på verdiene i features $\mathbf{x}$, altså $p(y|\mathbf{x})$. En modell som gjør dette gir oss *prediksjoner* $f(\mathbf{x}) = y_{\text{pred}}$.

1.2 Logistisk regresjon

Den enkleste modellen vi kan tenke oss er lineær regresjon,

$$z = \sum_{i=1}^n w_i x_i + b = \mathbf{w}\mathbf{x} + b, \quad (2)$$

hvor vektene w_i forteller oss hvor viktig hver feature x_i er for utfallet, altså verdien til z . En nyttig mental øvelse er å tenke over om z er et tall eller en vektor. Dette kan vi se basert på hvordan $\mathbf{w}$ og $\mathbf{x}$, som begge er vektorer, kombineres i uttrykket.

I likning 2 ser vi at z kan bli et hvilket som helst tall, mens vi er ute etter en sannsynlighet for overlevelse, altså et tall mellom 0 og 1. En enkel løsning på dette er å bruke den logistiske funksjonen $\sigma(x)$, også kalt Sigmoid-funksjonen fordi den er formet som en ‘S’, se figur 2. Denne mapper reelle tall til domenet $(0, 1)$:

$$\sigma(z) = \frac{1}{1 + e^{-z}}. \quad (3)$$

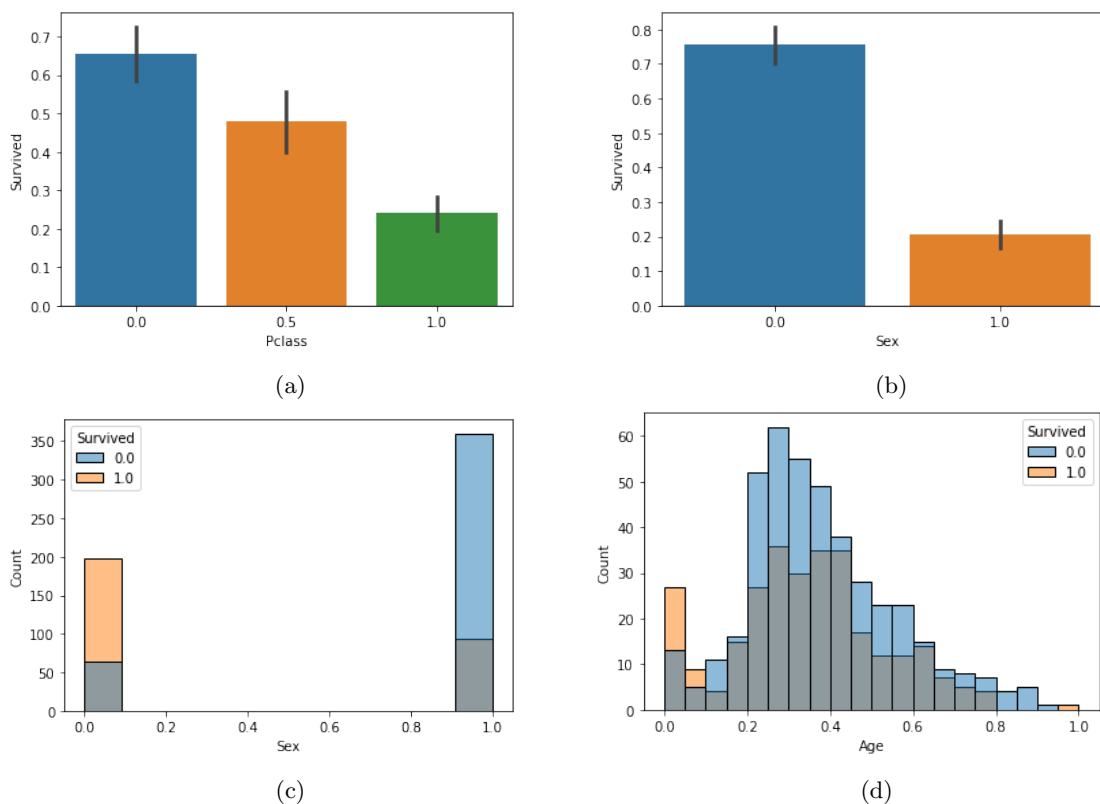


Figure 1: Bar-plot (a, b) og histogrammer (c, d) fra Titanic-dataene. Se beskrivelse i teksten.

Det kan være lurt å huske denne funksjonen, da vi kommer til å møte den igjen når vi diskuterer nevrale nettverk, da under navnet *aktiveringsfunksjon*. Vet å sette uttrykket i likning 2 inn i likning 3 får vi følgende uttrykk, som representerer modellens prediksjon for target y :

$$y_{\text{pred}} = \frac{1}{1 + e^{-\sum_i w_i x_i + b}}. \quad (4)$$

1.2.1 Trening og tapsfunksjon

Før en modell kan gi oss gode prediksjoner for $\mathbf{x}$ må den tilpasses til dataene våre, og denne prosessen omtales som *læring* eller *trening* innen maskinlæring. Læringsprosessen går ut på at modellen tilpasses slik at måloppnåelsen øker på datasettet vi bruker. I maskinlæring omtales data som brukes til dette som *treningsdata*, og funksjonen som brukes til å beregne måloppnåelse som *tapsfunksjon*, fra engelske *loss function*. Denne funksjonen forteller oss hvor nærme modellens prediksjon er den riktige verdien, altså target, og vi bruker følgende notasjon

$$\mathcal{L}(y_{\text{pred}}, y) \quad (5)$$

for å angi en tapsfunksjon $\mathcal{L}$ som tar inn modellprediksjonene y_{pred} og targets y for å beregne tapet til modellen på det aktuelle datapunktet.

Neste spørsmål er hvilken tapsfunksjon vi skal velge. Dette er avhengig av oppgaven vi ønsker at modellen skal løse, og alternativene er mildt sagt mange. I vårt tilfelle ønsker vi å maksimere sannsynligheten for riktig kategorisering (Survived=0 eller 1) av dataene. Vi kan tenke på dette som en sannsynlighet for at $y_{\text{pred}} = 1$ hvis $y = 1$ og for at $y_{\text{pred}} = 0$ hvis $y = 0$. Kompakt kan vi skrive dette som

$$y_{\text{pred}}^y (1 - y_{\text{pred}})^{1-y}. \quad (6)$$

Oppgave: Studer dette uttrykket. Hva er verdien hvis $y_{\text{pred}} = 1$ og $y = 1$? Hva med de andre kombinasjonene av de mulige verdiene til y_{pred} og y ?

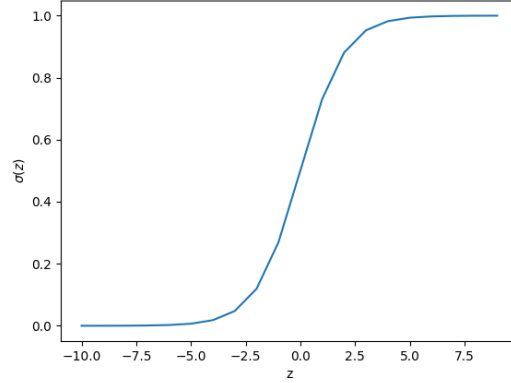


Figure 2: Sigmoid-funksjonen.

Vi har *ikke* lyst til å endre dataene slik at predikert sannsynlighet for overlevelse endrer seg. I stedet ønsker vi å endre modellen slik at den estimerte sannsynligheten for overlevelse passer med den virkelige. Vi er altså i den situasjonen at dataene er gitt, mens modellen kan endres (det er det vi gjør gjennom trening). Husk at modellen vår er parametrisert av $\mathbf{w}$ og b , og at $y_{pred} = \sigma(\mathbf{w}\mathbf{x} + b)$. Vi setter inn dette i likningen over og får

$$L(\mathbf{w}, b | \mathbf{x}, y) = \sigma(\mathbf{w}\mathbf{x} + b)^y (1 - \sigma(\mathbf{w}\mathbf{x} + b))^{1-y}. \quad (7)$$

Dette er en *likelihood* (derav L-en), og er en sannsynlighetsfordeling over modellparametre, gitt at data er observert. Hvorvidt vi snakker om likelihood eller sannsynlighet, er avhengig av hvilken situasjon vi er i: Hvis vi har en modell (parametrene er satt), kan vi bruke den til å estimere sannsynligheten for observasjoner (de da ikke-observerte x og y). Vi er derimot i den andre situasjonen; vi har observasjoner (dataene x og utfallene y), og ønsker å tilpasse en modell til disse. Da trenger vi en fordeling over parametrene, og som altså kalles en likelihood.

Uttrykket i likning (7) kan bli vanskelig å jobbe med på grunn av eksponentene. Siden vi kun ønsker å finne et optimum (minimum eller maksimum) i dette uttrykket, kan vi ta logaritmen, som er en monoton funksjon. Da har vi

$$\ln L(\mathbf{w}, b | \mathbf{x}, y) = \ln (\sigma(\mathbf{w}\mathbf{x} + b)^y (1 - \sigma(\mathbf{w}\mathbf{x} + b))^{1-y}) \quad (8)$$

$$= y \ln \sigma(\mathbf{w}\mathbf{x} + b) + (1 - y) \ln(1 - \sigma(\mathbf{w}\mathbf{x} + b)), \quad (9)$$

ofte omtalt som *log-likelihood*. Likelihood for å estimere riktig klasse bør *maksimeres*, så vi legger på et minustegn for å få en tapsfunksjon som skal *minimeres*. Da ender vi opp med såkalt *cross-entropy loss*:

$$\mathcal{L}(y_{pred}, y) = -\ln L(\mathbf{w}, b | \mathbf{x}, y) \quad (10)$$

$$= -y \ln \sigma(\mathbf{w}\mathbf{x} + b) - (1 - y) \ln(1 - \sigma(\mathbf{w}\mathbf{x} + b)). \quad (11)$$

Dette er tapsfunksjonen vi vil bruke for å tilpasse modellen til dataene.

Modellen vår er definert av parametrene $\mathbf{w}$ og b , som med kompakt notasjon kan skrives $\theta = (\mathbf{w}, b)$. Vi ønsker å finne parametrene som minimerer tapet, i gjennomsnitt over alle datapunktene. Dette kalles å *trene modellen*. Mer formelt vil vi velge parametrene θ som maksimerer log-likelihood for target-verdiene y for hvert datapunkt $\mathbf{x}$. Dette kalles *conditional maximum likelihood estimation*. For å trene modellen vår har vi altså følgende optimaliseringsproblem

$$\hat{\theta} = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f(\mathbf{x}^{(i)}; \theta), y^{(i)}). \quad (12)$$

For å løse dette optimaliseringsproblemet skal vi bruke *gradient descent*. Vi skal bruke den samme metoden senere i kurset når vi trener nevrale nettverk.

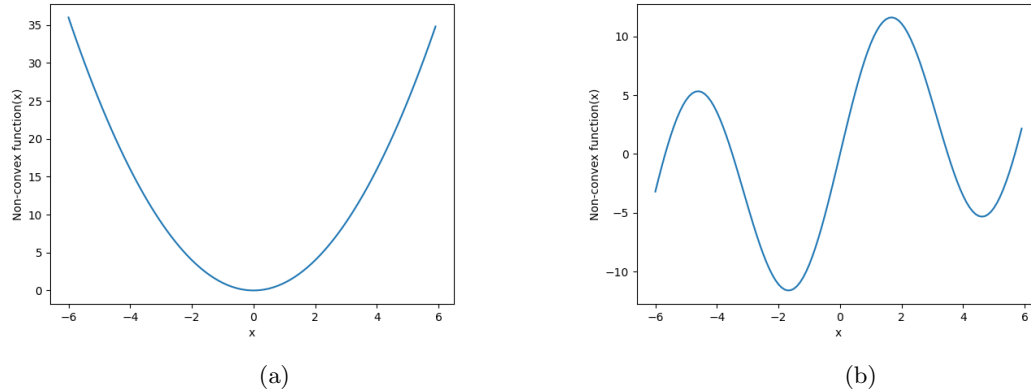


Figure 3: Eksempler på en (a) konveks, og en (b) ikke-konveks funksjon.

1.2.2 Gradient descent

Gradient descent er en metode for å finne minimumspunktet til en funksjon man ikke kjenner formen til. Strategien er å finne ut i hvilken retning i rommet definert av parametrene θ funksjonen minker brattest. Intuitivt kan vi se for oss at vi vandrer rundt på et fjell med bind for øynene, med mål om å finne det laveste punktet. Hvis man ikke kan se hvor det laveste punktet er, er beste strategi å føle seg frem til den bratteste nedoverbakken, og følge denne til man kommer ned fra fjellet.

En potensiell utfordring er at funksjonen kan ha både lokale og globale minima. Et lokalt minimum har en høyere funksjonsverdi enn et globalt minimum, men funksjonen har kun positive graderter rundt minimumspunktet. I bind-for-øynene-analogien svarer dette til at man befinner seg i en fjellkjede, og kan gå seg vill mellom fjelltopper uten å vite at det finnes en dypere dal bak neste fjelltopp. Tapsfunksjonen for logistisk regresjon er konveks, og hvilket betyr at den kun har ett optimum. Figur 3a viser et eksempel på en konveks funksjon, mens figur 3b viser en ikke-konveks funksjon, med flere minima. Så lenge tapsfunksjonen vår er konveks har vi en garanti om at gradient descent som finner et minimum har funnet et globalt minimum. **Oppgave:** Hvilken retning bør vi bevege oss i? I hvilket rom?

Rommet vi beveger oss i er parameterrommet, altså rommet spent ut av θ -verdiene; hvis vi har fire parametre, er rommet fire-dimensjonalt. Vi bør bevege oss i den retningen der tapet minker. Det vi trenger er altså et mål på hvordan $\mathcal{L}$ endrer seg som funksjon av endring i θ . Vi trenger den deriverte av $\mathcal{L}$ med hensyn på θ , altså *gradienten* (dette er gradienten i “gradient descent”). Her lærer vi noe viktig, som du bør huske til senere: Siden gradient descent er veldig utbredt innen maskinlæring, også til trening av nevralt nettverk, forstår vi at *tapsfunksjonen må være deriverbar*.

I starten av treningen har alle parametrene en tilfeldig verdi, og vi oppdaterer verdiene iterativt, gjennom flere steg. Gradienten til tapsfunksjonen peker i retning av økende tap, og siden vi ønsker å minke tapet må vi bevege oss i motsatt retning. Dette representerer vi gjennom et minustegn, og regelen for parameteroppdateringen kan skrives som følger

$$\theta^{t+1} = \theta^t - \eta \frac{\partial}{\partial \theta} \mathcal{L}(f(\mathbf{x}; \theta), y). \quad (13)$$

Den nye verdien θ^{t+1} etter steg t er den forrige verdien θ^t korrigert med gradienten til tapsfunksjonen. Parametere η representerer hvor store korrigeringer som skal gjøres til parameterverdien, og omtales derfor som *læringsraten*: en høy verdi av η medfører store korrigeringer, mens en liten η medfører små endringer. Vi vet at modellen defineres av parametrene θ . Læringsraten η er også en parameter vi kan justere for at treningen skal gå best mulig, men denne er ikke en del av modellen. Det er en såkalt *hyperparameter*. Hyperparametre er parametre som definerer læringsprosessen, men ikke modellen. Det er viktig å forstå forskjellen på (modell)parametre og hyperparametre.

Vi trenger et uttrykk for den deriverte av tapsfunksjonen med hensyn på modellens parametre, altså

hver w_i , og b . Det er en god øvelse å gjøre dette for hånd, og nyttige relasjoner er

$$\frac{\partial}{\partial x} \ln x = \frac{1}{x} \quad (14)$$

$$\frac{\partial}{\partial z} \sigma(z) = \sigma(z) (1 - \sigma(z)) \quad (15)$$

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial u} \frac{\partial u}{\partial x}. \quad (16)$$

Riktig uttrykk for de deriverte er

$$\frac{\partial \mathcal{L}}{\partial w_j} = \frac{\partial}{\partial w_j} [-y \ln \sigma(\mathbf{w}\mathbf{x} + b) - (1 - y) \ln (1 - \sigma(\mathbf{w}\mathbf{x} + b))] \quad (17)$$

$$= -y \frac{1}{\sigma(\cdot)} \frac{\partial}{\partial w_j} \sigma(\mathbf{w}\mathbf{x} + b) + (1 - y) \frac{1}{1 - \sigma(\cdot)} \frac{\partial}{\partial w_j} (1 - \sigma(\mathbf{w}\mathbf{x} + b)) \quad (18)$$

$$= \frac{-y(1 - \sigma(\cdot)) + (1 - y)\sigma(\cdot)}{\sigma(\cdot)(1 - \sigma(\cdot))} \frac{\partial}{\partial w_j} \sigma(\mathbf{w}\mathbf{x} + b) \quad (19)$$

$$= \frac{\sigma(\cdot) - y}{\sigma(\cdot)(1 - \sigma(\cdot))} \sigma(\cdot)(1 - \sigma(\cdot)) \frac{\partial}{\partial w_j} (\mathbf{w}\mathbf{x} + b) \quad (20)$$

$$= (\sigma(\mathbf{w}\mathbf{x} + b) - y) x_j \quad (21)$$

$$= (y_{\text{pred}} - y) x_j \quad (22)$$

$$\frac{\partial \mathcal{L}}{\partial b} = \dots \quad (23)$$

$$= (\sigma(\mathbf{w}\mathbf{x} + b) - y) \frac{\partial}{\partial b} (\mathbf{w}\mathbf{x} + b) \quad (24)$$

$$= (y_{\text{pred}} - y). \quad (25)$$

1.2.3 Trening

Vi har allerede kommet langt på vei for å gjøre maskinlæring: Vi har en modell, vi vet hvilke parametre som definerer den, vi har en tapsfunksjon, og vi har et uttrykk for å oppdatere parameterverdiene slik at tapet minker. Når vi har laget et program som utfører disse stegene, har vi laget en *læringsalgoritme*. En måte å strukturere et slikt program er gjennom følgende klasse og funksjonalitet (dere må skrive inn koden for funksjonene selv):

```
class LogisticRegression:
    def __init__(self, learning_rate=0.1, epochs=1000):
        self.learning_rate = learning_rate
        self.epochs = epochs
        self.weights, self.bias = None, None
        self.losses, self.train_accuracies = [], []

    def sigmoid_function(self, x):

    def _compute_loss(self, y, y_pred):

    def compute_gradients(self, x, y, y_pred):

    def update_parameters(self, grad_w, grad_b):

    def accuracy(true_values, predictions):
        return np.mean(true_values == predictions)
```

I tillegg trengs to viktige metoder: 1) **fit**, som tilpasser modellparametrene til treningsdataene. Her angir parameteren *epochs* hvor mange ganger gradient descent skal brukes på alle treningsdataene. 2) **predict**, som returnerer modellens prediksjon på data. Disse to navnene, **fit** og **predict**, er så standard i ulike maskinlæringsbiblioteker at man gjør hurt i å holde seg til dem og ikke dikte opp egne navn, selv når man skriver kode selv fra bunnen. Metodene kan for eksempel implementeres som følger


```

def fit(self, x, y):
    self.weights = np.zeros(x.shape[1]) #x.shape = datapunkter, features
    self.bias = 0

    # Gradient Descent
    for _ in range(self.epochs):
        lin_model = np.matmul(self.weights, x.transpose()) + self.bias
        y_pred = self._sigmoid(lin_model)
        grad_w, grad_b = self.compute_gradients(x, y, y_pred)
        self.update_parameters(grad_w, grad_b)

        loss = self._compute_loss(y, y_pred)
        pred_to_class = [1 if _y > 0.5 else 0 for _y in y_pred]
        self.train_accuracies.append(accuracy(y, pred_to_class))
        self.losses.append(loss)

def predict(self, x):
    lin_model = np.matmul(x, self.weights) + self.bias
    y_pred = self._sigmoid(lin_model)
    return [1 if _y > 0.5 else 0 for _y in y_pred]

```

Når koden for læringsalgoritmen er ferdig skrevet, er neste steg å ta i bruk dataene til å trene modellen. Dette bør vi *ikke* bruke hele datasettet til. Når vi er ferdige med å trene modellen har vi nemlig lyst til å undersøke hvor godt denne gjør det på data som ikke har blitt brukt til å tilpasse parametrene. Et sentralt mål innen maskinlæring er å lage modeller som klarer å *generalisere*, altså gjøre det godt på *nye* data. Derfor er det standard prosedyre å dele et datasett i minimum to deler, hvorav den ene delen er størst og utgjør *treningsdata*. Den andre, mindre delen representerer *testdataene*. Treningsdataene brukes til parametertilpasning, mens testdataene ikke røres før modellen er ferdig tilpasset, og brukes for å evaluere modellen før denne tas i bruk. For å dele datasettet i to kan metoden `train_test_split` fra biblioteket `scikit-learn` brukes.

Når læringsalgoritmen er på plass og dataene er splittet i en trenings- og en test-del, bør hele maskinlæringsprosedyren se ut omtrent som følger.

```

# Training
train_epochs = 30

# Initialize and train the model
log_reg = LogisticRegression_(learning_rate=0.01, epochs=train_epochs)
log_reg.fit(X_train, y_train)

# Make predictions
predictions = log_reg.predict(X_test)

```

1.2.4 Evaluering

Tenk på definisjonen av maskinlæring gitt i starten av kurset: For å vite at programmet faktisk blir bedre til å løse oppgaven gjennom erfaring (data), trenger vi en måte å måle ytelsen. Som et minimum bør vi lagre tapet (loss) og treffsikkerhet (accuracy) i løpet av treningen, og plote disse etter treningen. Dette kan gjøres med følgende kode, ved hjelp av biblioteket `matplotlib`,

```

epoch_list = np.arange(0, train_epochs, 1)
plt.plot(epoch_list, log_reg.losses, c='red', label="Loss")
plt.plot(epoch_list, log_reg.train_accuracies, c='blue', label="Accuracy")
plt.legend()
plt.show()

```

og et eksempel på resulterende plott fra en vellykket treningsprosedyre er vist i figur 4. På dette plottet ser vi at tapet er høyt i starten, og minker jevnt før det flater ut. Dette er bra; vi ønsker at

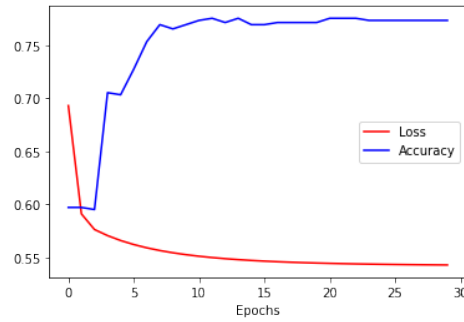


Figure 4: Eksempel på plott som viser tap (rød) og treffsikkerhet (blå) over epokene i en treningsprosedyre.

Table 4: Et utvalg metrikker vi kan beregne basert på verdiene i confusion matrix.

Betegnelse	Uttrykk	Alternative navn
True Positive Rate (TPR)	$\frac{TP}{P} = \frac{TP}{TP+FN}$	Sensitivitet, recall, sannsynlighet for deteksjon
True Negative Rate (TNR)	$\frac{TN}{N} = \frac{TN}{TN+FP}$	Spesifisitet
False Positive Rate (FPR)	$\frac{FP}{N} = \frac{FP}{FP+TN}$	Sannsynlighet for falsk alarm, (1 - spesifisitet)
Positive Predictive Value (PPV)	$\frac{TP}{TP+FP}$	Precision

tapet skal minke. Treffsikkerheten starter derimot lavt, og øker før den flater ut. Dette er også bra; vi ønsker økende treffsikkerhet. Til sammen gir disse to kurvene oss et bilde av en modell som lærer av dataene, og en treningsprosedyre som har foregått lenge nok. Hadde kurvene ikke flatet ut, hadde vi trengt flere epoker for at modellen skulle lære ferdig. I dette tilfellet kunne vi også stoppet treningen tidligere, etter rundt 20-25 epoker.

Selv om tap og treffsikkerhet kan fortelle oss om modellen lærer fra dataene, forteller de mildt sagt kun en liten del av historien, og gir oss ikke et godt bilde på hva modellen er god på eller hvilke typer datapunkter den typisk gjør feil på. De gir oss ikke en god anledning til å analysere modellens styrker og svakheter. Videre gir ikke tall som angir tap og treffsikkerhet intuitivt mening for mennesker. Vi bør derfor velge en eller flere *metrikker* for å angi modellens ytelse, og disse metrikkene bør gi mening for mennesker. I motsetning til tapsfunksjonen trenger ikke metrikken(e) vi velger å være deriverbar(e). **Oppgave:** Hva er flere grunner til at vi ikke bør velge samme tapsfunksjon og evalueringsmetrikk?

For klassifiseringsmodeller baseres mange metrikker på *confusion matrix*, hvis innhold angir hvor mange av datapunktene klassifiseres riktig (true) og feilaktig (false) i de ulike klassene. I vårt tilfelle har vi to klasser: 0 (negative), og 1 (positive), og vi kan sette opp en confusion matrix etter malen i figur 5. Her plasseres de sanne prediksjonene (true positive/negative), altså prediksjonene som samsvarer med target, på diagonalen, og de falske prediksjonene (false positive/negative) utenfor diagonalen i matrisen. Basert på verdiene i confusion matrix kan vi regne ut flere metrikker, for eksempel de angitt i tabell 4.

		Predicted	
		Negative (PN)	Positive (PP)
True	Negative (N)	True negative	False positive
	Positive (P)	False negative	True positive

Figure 5: Confusion matrix for binær klassifisering.

Oppgave: Du skal levere en maskinlæringsmodell til et sykehus som vil finne ut hvilke pasienter som lider av en dødelig sykdom. (I dette scenariet er det veldig alvorlig om modellen får en falsk negativ.) Hvilken metrikk bør modellen skåre høyt på? Senere skal du (du har mye å gjøre, ja) levere en modell

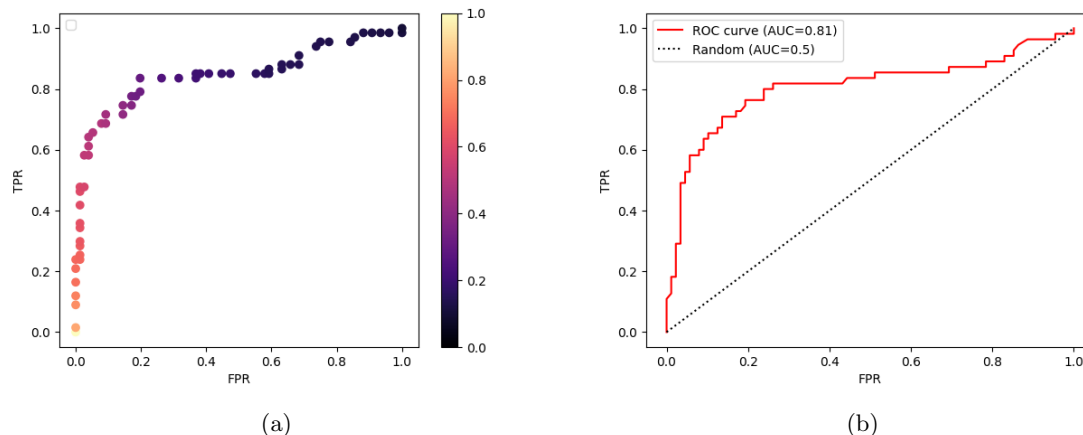


Figure 6: ROC-kurven, som viser FPR og TPR for ulike klassifiseringsterskler i $[0,1]$.

til klimaaktivister som vil skyte raketter på tomme passasjerfly. (I dette scenariet er det alvorlig om modellen får en falsk positiv, fordi raketter er dyre.) Hvilken metrikk bør denne modellen skåre lavt på?

1.2.5 Klassifiseringsterskel

Modellen vår returnerer et tall i intervallet $(0, 1)$, og en enkel måte å gjøre dette om til en klassifisering er det vi gjorde i koden for `fit` og `predict`, nemlig følgende linje

```
[1 if _y > 0.5 else 0 for _y in y_pred]
```

hvor vi tolker en prediksjon som `Survived=1` hvis `y_pred>0.5`, og ellers `Survived=0`. Her er tallet 0.5 klassifiseringsterskelen, og vi kunne i prinsippet valgt et annet tall. Hvis det var viktig for oss å være forsiktig, altså ikke predikere at en person vil overleve med mindre modellen er helt sikker, kunne vi valgt en terskel på for eksempel 0.7 i stedet. Hvilken terskel som bør velges er avhengig av problemet, og vi kan studere hvilket utslag ulike terskler har på ulike metrikker.

Det vanligste er å studere FPR og TPR som funksjon av klassifiseringsterskelen. Det gir oss Receiver Operating Characteristic (ROC)-kurven, se figur 6a. Her beregner vi både FPR og TPR for mange ulike klassifiseringsterskler fra 1 til 0. **Oppgave:** Ønsker vi høy eller lav verdi for henholdsvis FPR og TPR? Hva blir FPR og TPR for klassifiseringsterskel 0? Tilsvarende for 1?

Vi ønsker en høy TPR, altså sann positiv rate, mens vi ønsker at FPR, altså falsk positiv rate, holder seg lav. Hvis klassifiseringsterskelen er 1, vil nærmest ingen prediksjoner tolkes som positiv klasse, slik at både FPR og TPR er lave. Klassifiseringsterskel på 0 tilsier at nærmest alle prediksjoner tolkes som positiv klasse, hvilket vil gi høye verdier for både FPR og TPR. Derfor starter ROC-kurven i $(0, 0)$ og ender i $(1, 1)$. Det interessante er hva som skjer mellom disse punktene, særlig om TPR øker raskere enn FPR, og for hvilke klassifiseringsterskel som gir best trade-off mellom de to. **Oppgave:** Hvordan ser ROC-kurven ut for en modell som kaster mynt?

Uansett hvilken oppgave vi løser, ønsker vi en modell som har høy TPR og lav FPR: Des høyere TPR per FPR, des bedre er modellen, for alle terskler. Tilfeldig gjetning (myntkast) gir diagonalen $(0, 0) - (1, 1)$, den stiplede linjen i figur 6b. For å være bedre enn tilfeldig gjetning må kurven som representerer modellen derfor ligge over diagonalen, og arealet under kurven (forkortet AUC, fra area under curve) være større enn 0.5. Siden begge aksene går fra 0 til 1 er det største mulige arealet under kurven 1. Dette arealet, ROC AUC, representerer sannsynligheten for at modellen vil predikere en høyere verdi (nærmere 1 enn 0, for binær klassifisering) for et tilfeldig valgt positivt datapunkt enn for et tilfeldig valgt negativt datapunkt.

Utover ROC-kurven kan vi i prinsippet plote hvilke som helst metrikker for varierende klassifiseringsterskler for å studere modellens oppførsel. En annen vanlig kurve å studere er den som viser *precision*

og *recall*, se figur 7. Presisjon angir andel korrekt predikert positive per predikert positive, altså

$$\text{Precision} = \frac{TP}{TP + FP}. \quad (26)$$

Høy presisjon svarer til lavt relativt antall falske alarmer, siden TP da dominerer over FP . Recall angir andel korrekt predikert positive per totalt positive, altså

$$\text{Recall} = \frac{TP}{TP + FN}. \quad (27)$$

Høy recall svarer til lavt antall missed cases, siden TP da dominerer over FN .

Når vi justerer klassifiseringsterskelen ser vi at det finnes en tradeoff mellom precision og recall; når den ene øker, minker den andre. Hva som er en god balanse mellom de to, er avhengig av hva modellen skal brukes til. Hvis falsk alarm er dyrt (for eksempel fører til en utrykning), er høy presisjon viktig. Hvis kostnaden med å bomme på en positiv instans er høy (for eksempel diagnostisering av alvorlige sykdommer), er høy recall viktig. **Oppgave:** Når er presisjon = recall? Hvilken av disse metrikkene er avhengig av fordelingen mellom klassene?

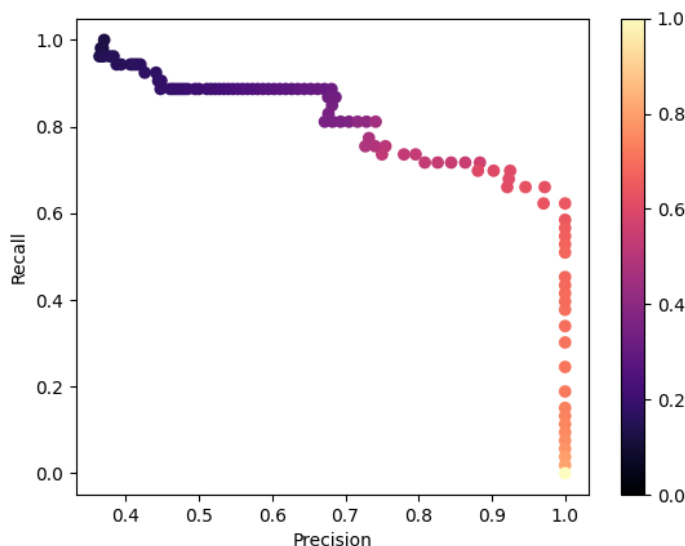


Figure 7: Precision-recall-kurven for ulike klassifiseringsterskler i $[0,1]$.

1.2.6 Kort om entropi og cross entropy loss

I informasjonsteori er entropi (også kalt Shannon entropy) en måte å kvantisere usikkerheten eller mengden informasjon forbundet med de mulige tilstandene x til en tilfeldig variabel X :

$$H(X) = \mathbb{E} \left[\frac{1}{\log p(X)} \right]. \quad (28)$$

Denne forventningsverdien kan vi beregne som vanlig,

$$H(X) = - \sum_{x \in X} p(x) \log p(x). \quad (29)$$

Vår tilgang til den tilfeldige variabelen X kommer fra fordelingen p som vi gjør trekninger fra, så vi kan skrive dette som

$$H(p) = - \sum_i p(x_i) \log p(x_i). \quad (30)$$

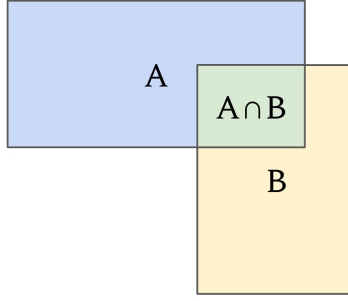


Figure 8: To delvis overlappende mengder A og B .

Altså: observasjonene kommer fra fordelingen p – alltid. I eksempelet med Titanic-dataene representerer p fordelingen av targets (**Survived**) i datasettet. I tillegg har vi en fordeling q , som representerer hva vi tror, det vil si hva modellen vår predikerer. Kryss-entropien (cross entropy) representerer da forskjellen mellom to ulike sannsynlighetsfordelingene, der p er fordelingen til de faktiske observasjonene (dataene), mens q er fordelingen av estimatene våre,

$$H(p, q) = - \sum_i p(x_i) \log q(x_i). \quad (31)$$

Se igjen på likning 28. Forventningsverdien er over faktiske observasjoner, mens fordelingen i logaritmen er det vi tror vi kommer til å observere. Sammenlikn så likning 31 med tapsfunksjonen i likning 10. Den ene sannsynlighetsfordelingen representerer labels i datasettet, mens den andre representerer modellens prediksjoner. Tapsfunksjonen forteller oss altså om *forskjellen* mellom de to fordelingene, og siden målet vårt er å lage en klassifiseringsmodell, ønsker vi at forskjellen mellom de to fordelingene er minst mulig. I tilfellet ikke-binær klassifisering, altså klassifisering til flere enn to klasser, generaliserer uttrykket i likning 31 til

$$\mathcal{L}(\hat{y}, y) = - \sum_i y_i \log(\hat{y}_i), \quad (32)$$

hvor summen går over alle klassene i .

1.2.7 Bayes' teorem

Se på de to delvis overlappende mengdene A og B i figur 8. Sannsynligheten for å trekke et element fra overlappet $A \cap B$ er

$$P(A \cap B) = P(A|B)P(B) = P(B|A)P(A). \quad (33)$$

Vi beholder den siste relasjonen, og omskriver den som følger

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}. \quad (34)$$

Dette uttrykket holder uansett sannsynlighetstolkning (frekventistisk eller Bayesiansk), og kalles Bayes' teorem. Når vi løser en klassifiseringsoppgave, som å beregne sannsynligheten for at en gitt passasjer overlever Titanic-ulykken, ønsker vi å estimere $p(C_k|x)$, altså sannsynligheten for klasse k (i vårt tilfelle kan k være 0 eller 1), betinget på en ekte observasjon $\mathbf{x}$. Dette kan vi bruke uttrykket over til. For enklere notasjon dropper vi vektornotasjonen på $\mathbf{x}$, dvs vi skriver x uten fet skrift, i dette delkapitlet. Vi skriver ut Bayes' teorem for å få et uttrykk for sannsynligheten vi er ute etter:

$$p(C_k|x) = \frac{p(x|C_k)p(C_k)}{p(x)}. \quad (35)$$

De ulike leddene i denne likningen betyr følgende

- $p(C_k)$ er *a priori* sannsynligheten for klasse, altså fordelingen av klassene i treningsdataene. Denne omtales som *prior*, fordi det er sannsynligheten for klassetilhørighet vi kan estimere uten å vite noe om det aktuelle datapunktet (legg merke til at x ikke forekommer i dette leddet).

- $p(x)$ er fordelingen av selve dataene, uavhengig av klasse (legg merke til at C_k ikke forekommer i dette leddet). Dette er den faktiske sannsynlighetsfordelingen av alle feature-verdiene for alle dataene, og den har like mange dimensjoner som datasettet har features. Dette leddet omtales som *evidens*.
- $p(x|C_k)$ er den betingede sannsynlighetsfordelingen av dataene gitt klassen, og det er en *likelihood*. Det er altså en egen sannsynlighetsfordeling – med like mange dimensjoner som datasettet har features – for hver klasse, i vårt tilfelle to.

Når vi bruker maskinlæring, eller en hvilken som helst form for dataanalyse, til å estimere klassen til et datapunkt, er det nettopp $p(C_k|x)$ vi prøver å estimere. Gitt likning 35, vet vi at det finnes et *analytisk* uttrykk for $p(C_k|x)$. Hvis vi klarer å beregne alle leddene i likning 35 trenger vi derfor ikke å estimere sannsynligheten; vi kan i prinsippet bare beregne den nøyaktig. Hvorfor gjør vi ikke det?

La oss se på leddene i likning 35 for Titanic-dataene. Kan vi beregne alle leddene nøyaktig?

- $p(C_k)$ kan beregnes: Det er antallet henholdsvis Survived=0 og Survived=1 per totalt antall datapunter. Hvis 50% av passasjerene overlever, er dette leddet lik 0.5.
- $p(x)$ er problematisk å skrive ned, siden vi trenger informasjon om alle mulige verdier av alle featurene i datasettet, og det videre ikke er sikkert at vi vil ende opp med en funksjon med et analytisk uttrykk. Det kan derfor bli vanskelig å finne den faktiske verdien til sannsynligheten for et gitt datapunkt x . Men: siden dette leddet er uavhengig av klasse, vil det ha samme verdi for begge klassene. Så lenge vi bare er ute etter sannsynligheten for en klasse gitt data, kan vi derfor ignorere det, da det bidrar like mye til begge klassene.

Vi dropper $p(x)$, og gjør om sannsynligheten for klasse til følgende proporsjonalitet

$$p(C_k|x) \propto p(x|C_k)p(C_k). \quad (36)$$

Nå gjenstår kun leddet som representerer likelihood for data gitt klasse:

- $p(x|C_k)$ er utfordrende å beregne så lenge features x er avhengige av hverandre, siden vi da må finne et uttrykk for en flerdimensjonal simultanfordeling. Dette er vanskelig (kanskje umulig) på samme måte som for evidensen $p(x)$.

En løsning som noen ganger fungerer i praksis, er å anta *uavhengige features*. For eksempel kan en passasjer være kvinne eller mann uavhengig av alder; verdien til feature “Sex” er *uavhengig* av feature “Age”. Så lenge dette holder for alle features, kan vi skrive om simultanfordelingen til et produkt av fordelinger over hver enkelt feature, og estimatet vårt forenkles til

$$p(C_k|x) \propto p(C_k) \prod_{i=1}^n p(x_i|C_k). \quad (37)$$

Dette kalles *Naïv Bayes*, fordi det er naïvt å behandle features som uavhengige (selv om det i noen tilfeller kan stemme), og den tilsvarende sannsynligheten for klassetilhørighet er

$$\hat{y} = \arg \max_{k \in \{1, \dots, n\}} p(C_k) \prod_{i=1}^n p(x_i|C_k). \quad (38)$$

I forelesningene går vi igjennom et eksempel hvor vi estimerer sannsynlighet for overlevelse for to ulike tilfeller 0 og 1, og finner ut at denne er mindre for tilfelle 0 enn for tilfelle 1, altså

$$p(C_1) \prod_{i=1}^n p(x_i|C_1) > p(C_0) \prod_{i=1}^n p(x_i|C_0). \quad (39)$$

Konklusjonen blir da at sannsynligheten for å overleve er større enn sannsynligheten for å dø, for det aktuelle datapunktet. Denne sannsynligheten er ikke kalibrert, så den er *ikke* en ekte posterior, men så lenge vi kun er ute etter å finne ut det relative forholdet mellom to sannsynligheter og beregner begge på samme måte (og antakelsene våre holder!), er vi good.

1.2.8 Ikke-uavhengige features

Om vi studerer featurene i Titanic-dataene ser vi at enkelte features sannsynligvis ikke er uavhengige.

Naïv Bayes er fint så lenge antakelsen om uavhengige features holder, i det minste tilnærningsvis. Men hva om features ikke er uavhengige? La oss gå tilbake til uttrykket fra likning 35, gjentatt under,

$$p(C_k|x) = \frac{p(x|C_k)p(C_k)}{p(x)}. \quad (40)$$

Vi har fordelingen av klassene fra datasettet, så $p(C_k)$ klarer vi å estimere. Fordelingen av dataene, $p(x)$ er vanskeligere (og kanskje umulig), men siden den er uavhengig av klasse er den bare en skalering av sannsynlighetsestimatet. Om vi kun ønsker å finne ut om en passasjer, representert av x , forventes å overleve eller ikke, kan vi se på forholdet

$$\frac{p(x|C_0)p(C_0)}{p(x|C_1)p(C_1)}. \quad (41)$$

Det vi trenger å estimere er altså simultanfordelingen av features betinget på hver klasse. Konkret i vårt lille Titanic-tilfelle altså $p(\text{Sex}, \text{Age}, \text{Pclass} | \text{Survived} = 0)$ og $p(\text{Sex}, \text{Age}, \text{Pclass} | \text{Survived} = 1)$.

La oss anta at vi har tre kontinuerlige features, i stedet for Titanic-tilfellet der både Sex og Pclass er kategoriske, og at vi har lyst til å estimere simultanfordelingen til disse tre. Vi klarer ikke å finne et analytisk uttrykk for denne simultanfordelingen, men vi prøver oss altså på et estimat. Vi kan lage en tabell, men det er lettere å se for oss et histogram (som er en slags visualisering av en tabell). La oss si at vi tenker at 10 bins er stor nok oppløsning per akse (altså per feature), og at vi krever 10 datapunkter per bin.

Oppgave: Hvor mange datapunkter trenger vi da for tre features? Hva er sammenhengen mellom antall features, antall bins og antall datapunkter?

Du kommer nok frem til at dette er en simultanfordeling som *beregningsmessig* lar seg estimere, siden vi bare trenger 10,000 datapunkter.

1.2.9 Dimensjonsforbannelsen

Om vi ønsker å estimere en d -dimensjonal fordeling, er antallet datapunkter som må samles inn avhengig av d . For å få en intuitjon kan vi tenke oss at vi igjen estimerer fordelingen ved hjelp av et histogram med 10 bins og der vi er fornøyde med 10 datapunkter per bin.

Oppgave: Hvordan skalerer sammenhengen mellom dimensjoner (i vårt tilfelle antall features) og antall datapunkter vi trenger?

Tabell 5 viser antall nødvendige datapunkter for fire ulike verdier av d . Vedien $d = 784$ er tatt med fordi dette er dimensjonaliteten til det mye brukte datasettet MNIST, som består av håndskrevne sifre i sort-hvitt (gjør gjerne sjekk det ut). Til sammenlikning er det $\sim 10^{82}$ atomer i universet.

Table 5

Dimensjoner d	Bins	Datapunkter
1	10	10^2
3	10^3	10^4
10	10^{10}	10^{11}
784	10^{784}	10^{785}

1.2.10 Trening på ubalanserte data

En masterstudent blir invitert til jobbintervju som ML-utvikler. Under intervjuet får kandidaten følgende spørsmål: “Vi har et datasett fra det norske helsevesenet, og skal lage en modell som klassifiserer friske og syke personer. Modellen vår får en treffsikkerhet (accuracy) på 99%, men den klarer nesten ikke å identifisere syke personer. Hva har skjedd?” **Oppgave:** Hva vil du svare på dette?

Dette vil typisk skje om den ene klassen (for eksempel den som representerer syke personer) inneholder mange færre datapunkter enn den andre. Førstnevnte klasse er da *underrepresentert*, mens klassen med flere datapunkter er *overrepresentert*. Hvis forskjellen mellom antall datapunkter i de to klassene er veldig stor, vil modellen belønnes om den bare predikerer at alle datapunktene tilhører den overrepresenterte klassen. Dette kalles også *prior probability shift*, siden modellprediksjonen domineres av $p(C_k)$ i likning 35, og ikke av leddene som inneholder informasjon om feature-verdiene. Modellen vil slik oppnå høy treffsikkerhet, men likevel ikke være det minste nyttig. Mens en slik modell vil ha en høy treffsikkerhet og en høy ROC AUC, vil andre metrikker være lave. I dette tilfellet, der positiv klasse er underrepresentert, vil recall vil være oppsiktsvekkende lav. Dette illustrerer at det er viktig å studere flere ulike metrikker for å forstå modellens svakheter. Videre forteller det oss at vi må gjøre noe mer for å få en modell til å bli nyttig når vi har skjevfordeling mellom klassene i (trenings-)datasettet.

Den enkleste tilpasningen vi kan gjøre er å endre klassifiseringsterskel i uttrykket

```
y_pred = [1 if _y > 0.5 else 0 for _y in y_pred]
```

fra 0.5 til en verdi som gir modellen høyere verdi på de andre metrikkene. Terskelen kan for eksempel bestemmes basert på precision-recall-plottet. Ved å senke terskelen kan vi øke andelen predicted positive, fordi alt over terskelen predikeres som 1. Å maksimere én metrikk går som oftest på bekostning av andre metrikker, og vi må huske at å justere denne terskelen *ikke* forbedrer modellen: det endrer kun hvordan vi forholder oss til modellens prediksjoner. Utover å endre terskelen kan vi også justere dataene eller tapsfunksjonen.

Resampling

Når vi gjør resampling for å gå fra et skjevfordelt til et jevnt fordelt datasett, har vi to muligheter.

- **Undersampling:** Her trekker vi like mange datapunkter fra den overrepresenterte klassen som vi har tilgjengelig i den underrepresenterte klassen. Da ender vi opp med et datasett bestående av like mange datapunkter fra hver klasse, men potensielt veldig få datapunkter totalt. Dette kan føre til at modellen som trenes på dataene undertilpasser (underfit).
- **Oversampling:** Her kopierer vi instanser fra den underrepresenterte klassen, inntil vi har like mange datapunkter fra den underrepresenterte som fra den overrepresenterte klassen. Da ender vi også opp med like mange datapunkter fra hver klasse, men potensielt mange duplikater fra den underrepresenterte klassen. Dette kan føre til at modellen som trenes på dataene overtilpasser (overfit).

Disse to teknikkene kan naturligvis kombineres, altså at man både undersampler den overrepresenterte klassen og oversampler fra den underrepresenterte klassen, slik at man til sammen har balanserte klasser i det resulterende datasettet. **Oppgave:** Vi har et datasett med 100 datapunkter fra den ene klassen, og 1000 fra den andre. Hvor mange datapunkter får vi totalt om vi gjør kun undersampling? Hvor mange får vi totalt om vi gjør kun oversampling, og hvor mange duplikater får vi?

Vektet tapsfunksjon

I stedet for å endre hvilke data modellen får tilgang til, kan vi fortelle modellen at en av klassene (ofte den underrepresenterte) er ekstra viktig. Dette gjør vi gjennom å straffe feilprediksjoner (gi høyere tap) på den viktigste klassen høyere relativt til de(n) andre klassen(e). Se igjen på tapsfunksjonen binary cross entropy fra likning 10:

$$\mathcal{L}(y_{\text{pred}}, y) = -y \ln y_{\text{pred}} - (1 - y) \ln(1 - y_{\text{pred}}) \quad (42)$$

Hvordan kan vi tilpasse denne slik at klassene vektes ulikt?

Hvis target $y = 0$ er første ledd null, og bidrar ikke til tapet. Kun andre ledd bidrar til tapet, som betyr at en vektning av andre ledd i likning 10 vil øke tapet for datapunkter med $y = 0$. Det samme gjelder for target $y = 1$: her er andre ledd null, og bidrar ikke til tapet. Vektning av det første leddet vil da føre til høyere loss for klassen med label $y = 1$. Den vektete tapsfunksjonen blir derfor

$$\mathcal{L}(y_{\text{pred}}, y) = -w_1 y \ln y_{\text{pred}} - w_0 (1 - y) \ln(1 - y_{\text{pred}}), \quad (43)$$

hvor w_0 og w_1 representerer vekter for klasser 0 og 1. Hvis verdien på w_0 er betydelig større enn verdien på w_1 vil modellen prioritere å predikere riktig verdi for datapunkter med label $y = 0$, og vice versa.

1.3 Beslutningstrær

1.3.1 Noder

Beslutningstrær hjelper oss å ta beslutninger for datasett ved å splitte dataene i noder som til sammen utgjør en trestruktur, se figur 9a. *Rotnoden* (root node) er starten på treet, der data med alle features kommer inn. Under rotnoden finner vi *beslutningsnoder* (decision nodes). Rotnoden og beslutningsnodene har kriterier for å splitte dataene (splitting criteria). Beslutningsnoder er alle noder under rotnoden som har ett eller flere splitting criteria. Nederst i treet finner vi *løvnoder* (leaf nodes), som ikke splitter dataene. De inneholder predikert verdi for datainstansen som ble sendt gjennom treet. Beslutningstrær består av *trestumper* (tree stumps), se figur 9b. En trestump består av én rotnode og n løvnoder, for n mulige utfall. Vi holder oss til tilfellet $n = 2$, altså at to mulige utfall etter hver splitt. Hver beslutningsnode splitter dataene på én feature, gjennom et såkalt splitt-kriterium. Det samme treningsdatasettet og labels kan gi opphav til mange ulike beslutningstrær.

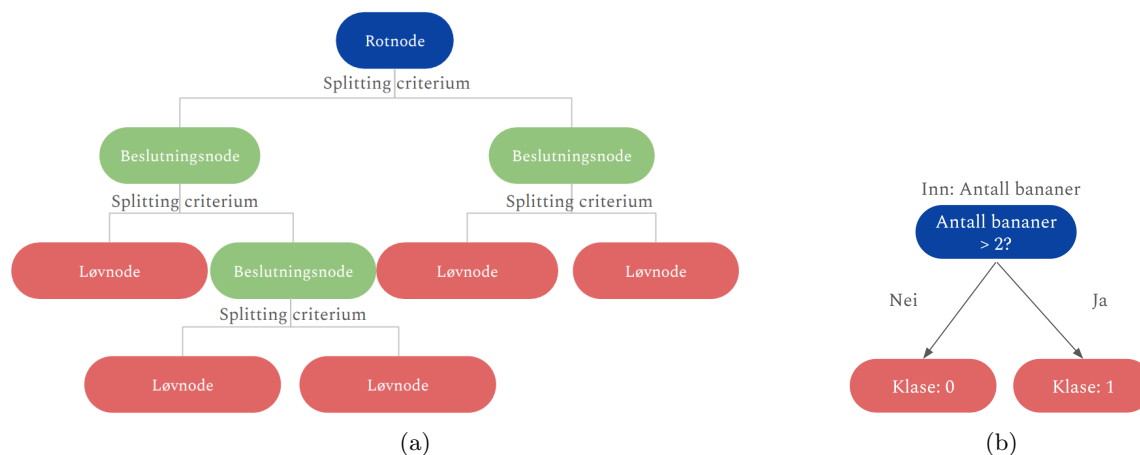


Figure 9: Skisser av (a) et beslutningstre, og (b) en stump.

Som ellers i maskinlæring, bruker vi treningsdata for å bygge modellen, i dette tilfellet beslutningstreet. Treningsdataene brukes for å finne ut hvilke trestumper (og tilhørende beslutningskriterier) som bør settes sammen for å lage treet. Når vi bygger beslutningstrær ønsker vi alltid å velge det splitt-kriteriet som lar oss ta beslutningen tidligst mulig, altså reduserer usikkerheten mest mulig. Redusert usikkerhet er endringen i usikkerhet etter sammenliknet med før splitt. I hovedsak brukes følgende tre metrikker for å måle hvor mye et splitt-kriterium (feature og verdi) reduserer usikkerheten:

- Log loss
- Gini impurity
- Entropi

Den første av disse, log loss, kjenner vi fra før, se likning 10 for binær klassifisering og likning 32 for multiklasse.

1.3.2 Gini Impurity

Gini-urenheten er et tall i $[0, 0.5]$ som angir sannsynligheten for at et nytt, tilfeldig datapunkt feilklassifiseres hvis det gis et tilfeldig label i henhold til klassedistribusjonen i datasettet. Gitt et datasett D bestående av datapunkter fra k klasser, med sannsynlighet p_i for at en instans tilhører klassen i ved en gitt node, er datasettets Gini-urenhet

$$\text{Gini}(D) = 1 - \sum_{i=1}^k p_i^2. \quad (44)$$

Intuitivt: Du har en pose med kuler i ulike farger, hvor farge representerer klasse. Gini-urenheten måler hvor sannsynlig det er at du gjetter feil farge på en tilfeldig trukket kule, hvis du gjetter at kulens farge følger distribusjonen av farger i posen. Lav Gini-urenhet representerer scenariet der de fleste kulene har samme farge, slik at det er *lav* sannsynlighet for å gjette feil farge på en tilfeldig trukket kule. Datasettet regnes da å ha lav urenhet. Høy Gini-urenhet representerer motsatt scenario, der klassene er forholdsvis likt representert, og det er *høy* sannsynlighet for å gjette feil farge på en tilfeldig trukket kule. Datasettet regnes da å ha høy urenhet.

Hvis et dataset D splittes på feature f til to subsett D_1 og D_2 med henholdsvis n_1 og n_2 datapunkter, har vi

$$\text{Gini}_f(D) = \frac{n_1}{n} \text{Gini}(D_1) + \frac{n_2}{n} \text{Gini}(D_2). \quad (45)$$

Table 6: Eksempler på Gini-urenhet for ulike splits.

	Antall		Sannsynlighet		Gini
	n_1	n_2	p_1	p_2	$1 - p_1^2 - p_2^2$
Alt A	0	10	0	1	$1 - 0^2 - 1^2 = 0$
Alt B	3	7	0.3	0.7	$1 - 0.3^2 - 0.7^2 = 0.42$
Alt C	5	5	0.5	0.5	$1 - 0.5^2 - 0.5^2 = 0.5$

1.3.3 Entropi

Gitt en sannsynlighetsfordeling over k klasser, er sannsynligheten for hver klasse p_i . Entropien til fordelingen er

$$\text{I}(p_1, \dots, p_k) = - \sum_{i=1}^k p_i \log_2(p_i). \quad (46)$$

Den maksimale entropien til en binær fordeling er 1, som svarer til lik sannsynlighet per klasse. Den minimale entropien er 0, som svarer til at alle datapunktene tilhører samme klasse.

Før vi splitter på en feature har vi et datasett D med p positive og n negative instanser. Dette svarer til en binær distribusjon der positiv klasse har sannsynlighet $\frac{p}{p+n}$, og negativ klasse har sannsynlighet $\frac{n}{p+n}$, og entropien er

$$\text{I}_{\text{før}} = \text{I}(p_{\text{positiv}}, p_{\text{negativ}}) = \text{I}\left(\frac{p}{p+n}, \frac{n}{p+n}\right). \quad (47)$$

Etter at treet splitter på en feature, har vi to nye datasett, D_1 og D_2 (ett på hver side av splitten). Den forventede entropien etter denne splitten er generelt

$$\mathbb{E}[\text{I}_{\text{etter}}] = \sum_{i=1}^s \frac{p_i + n_i}{p+n} \text{I}\left(\frac{p_i}{p_i + n_i}, \frac{n_i}{p_i + n_i}\right). \quad (48)$$

hvor s angir antall splitt, i vårt tilfelle $s = 2$, slik at $i = 1, 2$. Vi er interessert i forskjellen i entropi, altså entropien før og etter en valgt splitt:

$$\Delta \text{I} = \text{I}_{\text{før}} - \mathbb{E}[\text{I}_{\text{etter}}] = \text{I}\left(\frac{p}{p+n}, \frac{n}{p+n}\right) - \sum_{i=1}^s \frac{p_i + n_i}{p+n} \text{I}\left(\frac{p_i}{p_i + n_i}, \frac{n_i}{p_i + n_i}\right), \quad (49)$$

og vi bør velge den splitten som gir størst endring i entropi.

1.3.4 Bygge beslutningstrær

I dette kurset trenger dere ikke bygge beslutningstrær selv, men dere bør kunne angi pseudokode for å bygge et beslutningstre. For å lage beslutningstrær bruker vi som oftest biblioteket **sklearn**, med den innebygde **DecisionTreeClassifier**. Denne lar oss velge mellom de splitt-kriteriene Gini impurity, entropy og log loss.

Gini impurity og entropi oppfører seg likt, og vil i hovedsak gi opphav til samme feature splits. Hovedforskjellen mellom de to ligger i at beregning av entropi krever en ekstra logaritme, som koster mer å beregne. Gini impurity er derfor mindre beregningstungt, altså raskere å regne ut. Denne er også default i **sklearn.DecisionTreeClassifier**.

Utover valg av beslutningskriterier, må vi besvare to ytterligere spørsmål:

- Når skal vi slutte å splitte, selv om nederste node har instanser fra begge klassene?
- Hva gjør vi med eventuelle løvnoder som har instanser fra begge klassene?

Tre tilfeller kan oppstå der vi må slutte å splitte: Når

1. det ikke finnes flere features å splitte på. Vi kan havne i en situasjon der vi har splittet på alle tilgjengelige features, men fremdeles ikke har klart å lage løvnoder som tilordner alle treningsdatapunktene til riktig klasse. Hvis vi ikke har flere features å splitte på, kan vi ikke lage en beslutningsnode: da er inneværende node nødvendigvis en løvnode, selv om den inneholder datapunkter fra flere klasser,
2. det ikke finnes flere datapunkter å teste. Vi kan havne i en situasjon der alle kombinasjoner av features som er *tilgjengelig i dataene* har blitt testet, uten at alle tenkelige kombinasjoner av features er testet,
3. treet har nådd en predefinert maksimal dybde. Dette er en hyperparameter som velges før man begynner å bygge treet.

Etter at treet er bygget vil vi sannsynligvis ha løvnoder som inneholder treningsdatapunkter fra begge klasser. For å bestemme hvilken beslutning en slik node kan ta har vi flere muligheter, hvorav de vanligste er å returnere

1. den dominante klassen i noden, altså label tilsvarende den dominante klassen fra treningsdataene i løvnoden,
2. et tilfeldig trukket label fra treningsdataene, altså *a priori*-sannsynligheten.

Vi kan nå sette sammen alt vi har vært gjennom til en algoritme som rekursivt bygger et beslutningstre, se pseudokode under.

```
def build_decision_tree(data, features, depth):
    if (all data in same class):
        return class label
    elif (no features left to test) or (depth >= max_depth):
        return 1 if p/(p+n) > n/(p+n) else 0
    elif (no data left to test):
        return (dominant class in parent node)
    else:
        choose best_feature f for split
        partition data based on f
        for each data partition:
            build_decision_tree(data partition, features - best_feature, depth+1)
```